

R1

Pierson Lipschultz

March 24, 2026

1 New Code / Devs

1.1 Python Terminal Script for Filtering

Because we're likely to need to filter quite a few systems with very similar methods I decided to make an actual script instead of just using a notebook. The largest upside is that this can easily be run on a compute node, which should make filtering larger sims easier.

Takes a user input in the CLI (see below), filter for all BH_Sol systems, and returns the population file with said system as well as a .csv of just the BH_Sol rows. Furthermore, as it is running it provides an output via terminal stating how many BH_MS systems it found, how many are S1:S2, as if any are S2:S1, it prints a warning as well as listing the count of them.

Usage Example

```
python BH-Sol_Filter.py /Data/1e+00_Zsun_population.h5 -o ./testResults
```

Arguments

Argument	Use	Ex value
-o	output location	.results
-outputName	name for the outputed files ¹	BH_Sol_Systems
-overwrite	Whether to overwrite existing results, T/F bool	True
--maxMass	Max mass used when filtering, defaults to $3M_{\odot}$	3.5
--maxPeriod	Max period used when filtering, defaults to 3.5 days	4

Can be found at https://github.com/PiersonLip/stellarBHs/blob/main/FilteringScripts/BH-Sol_Filter.py

¹If a name is not specified, it will use the original filename, and append a specifier to the front

2 New Research

When filtering via notebook and script, both returned 6 systems that had $p_{orb} < 3.5, M_{\odot,2} < 3, t < 10e9$

These systems all evolved via the same channel, that being ZAMS_oCE1_CC1_oRL02_CC2. Via inspection of the history df and the oneline, it is evident that the systems evolve detached, then post-MS **unstable** RLO happens via $S1 \rightarrow S2$. This leads to common envelope, which *greatly* decreases the orbital period. This envelope is then ejected, leaving a depleted stripped He core. The core then collapses in a couple thousand years, leaving the system in a BH-MS state with low p_{orb} /seperation.

These systems then evolve in a semi-peculiar way, where it is highly likely that there will be RLO from S2 onto S1, and because $S1 \nless S2$, this is stable, and leads to *very* prolonged stable RLO (seemingly on the scale of $> 14Gyr$), which leaves S2 at low masses ($< .08M_{\odot}$).

Interestingly, the initial period for all the systems is very close to $4000d$, with mean of 4022 and a std of only 151. However, a larger sample size is needed to confirm this.

Additionally, the initial masses of S1 and S2 appear to be quite constrained as well.

	orbital_period.i		S1_mass.i		S2_mass.i
count	6.000000	count	6.000000	count	6.000000
mean	4022.716197	mean	17.515761	mean	2.296788
std	151.522384	std	0.520269	std	0.320702
min	3873.118288	min	16.815111	min	1.797809
25%	3908.300983	25%	17.190036	25%	2.120753
50%	3978.305326	50%	17.510176	50%	2.353828
75%	4124.397101	75%	17.818155	75%	2.547210
max	4248.507680	max	18.254965	max	2.620529

2.1 Bugs/issues

Found an issue with the script used to generate the slurm jobs, fails due to a *lack of* a var being passed

For example, when running

```
posydon-popsyn setup BH_Sol_Grid2.ini --job_array=50 --walltime=40:00:00  
  
--partition=ciera-std --account=b1094 --email=piersonlipschultz@gmail.com --mem_per_cpu=12G
```

Each slurm job array fails from...

Srun fatal: SLURM_MEM_PER_CPU, SLURM_MEM_PER_GPU and SLURM_MEM_PER_NODE are mutually exclusive

This isn't necessarily a posydon issue, as it happens when the script is being already ran *in* an interactive compute node, as the memory env then gets passed to the script. Fixed it with `srun --mem=0` or just not running in an interactive node.

3 Questions

- Tried running a grid, seems to have just completely failed from trying to reference itself before it was fully initialized. Entire log was filled with

```
Traceback (most recent call last):
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/popsyn/binarypopulation.py", 1
    binary.evolve()
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/binarystar.py", 1
    self.run_step()
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/binarystar.py", 1
    next_step(self)
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/DT/step_merged.py", 1
    super().__call__(binary)
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/DT/step_isolated.py", 1
    super().__call__(binary)
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/DT/step_detached.py", 1
    self.update_after_evo(t, binary, primary, secondary)
File "/gpfs/projects/b1095/bku2126/UCXBInvestigation/POSYDON/posydon/binary_evol/DT/step_detached.py", 1
    current = history[-1]
               ~~~~~~
IndexError: index -1 is out of bounds for axis 0 with size 0
```

and from what I can tell, this happened to nearly all the simed binaries

- Should we include an initial eccentricity distribution? I imagine it would heavily impact formation with unstable RLO and CE